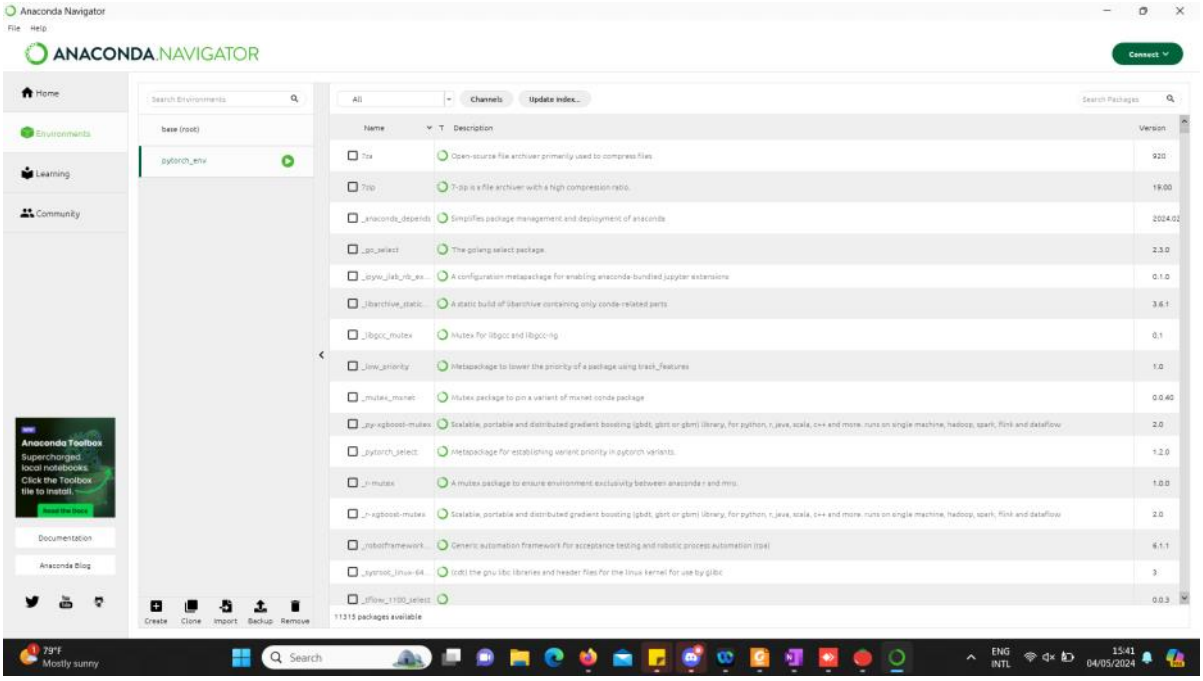
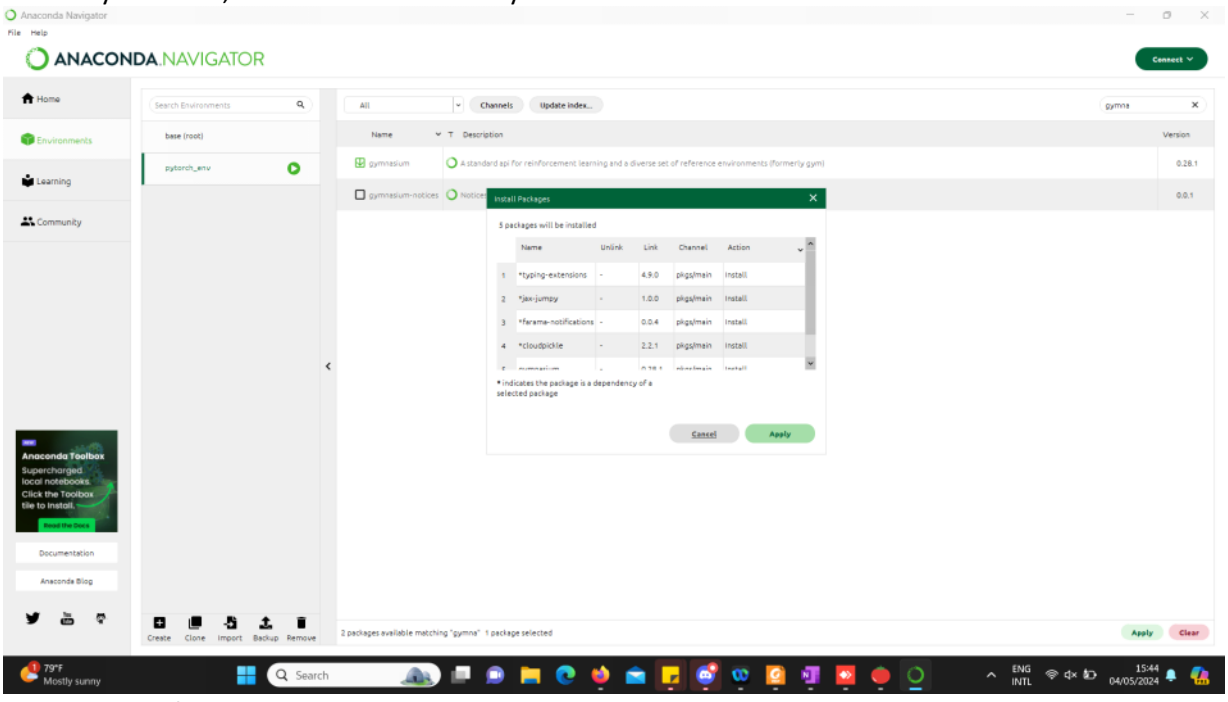
Chapter 1 Experiment

03 May 2024 15:09

1. Deploy the environment for the experiment
    a. Install Anaconda
    b. Install pythorch with Anaconda, you can using this tutorial: https://saturncloud.io/blog/a-guide-to-installing-pytorch-with-anaconda-and-troubleshooting-errors/. There are some important parts in this tutorial: conda install pytorch torchvision cudatoolkit -c pytorch alternatively you can install via anaconda-navigator GUI

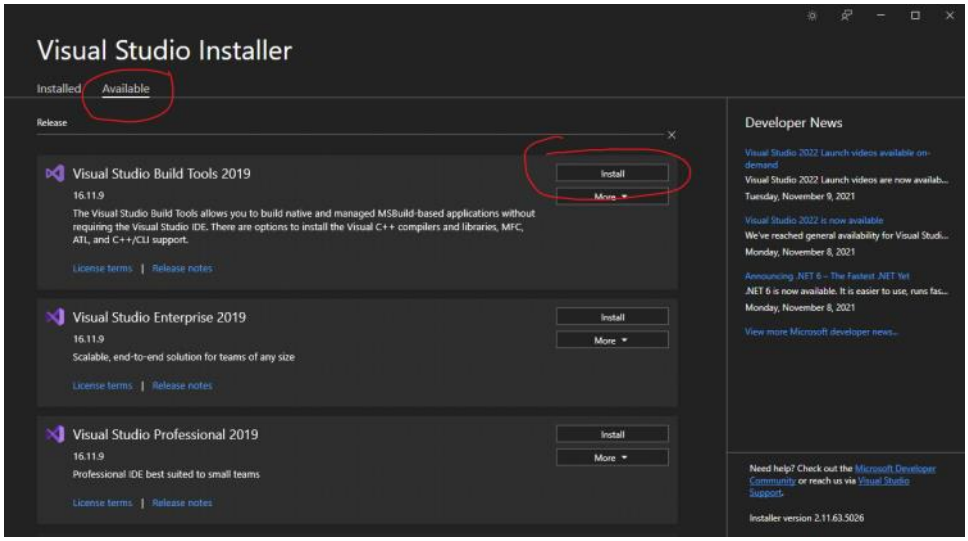


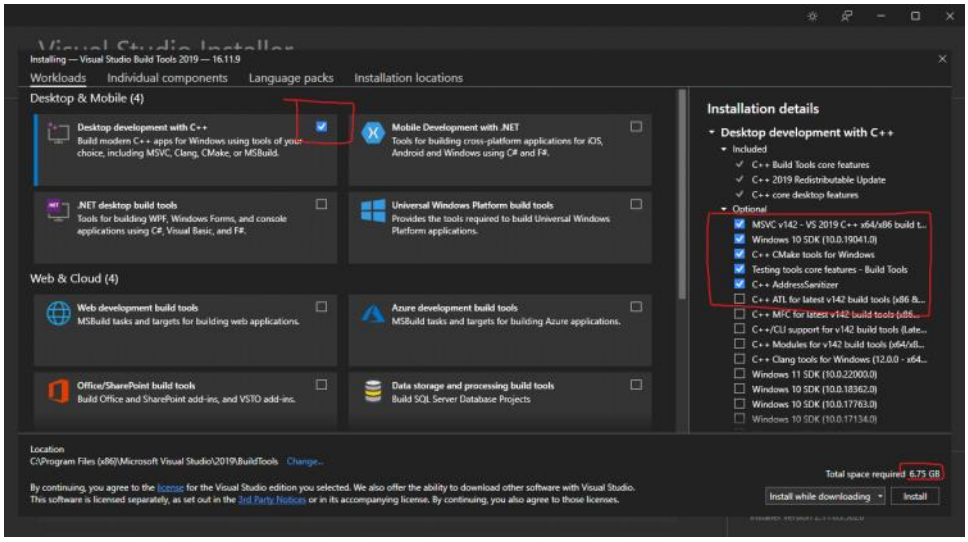
    c. Install Gymnasium, the environment library.



    d. Instal Microsoft Visual Studio Build Tools
        i. Install Visual Studio

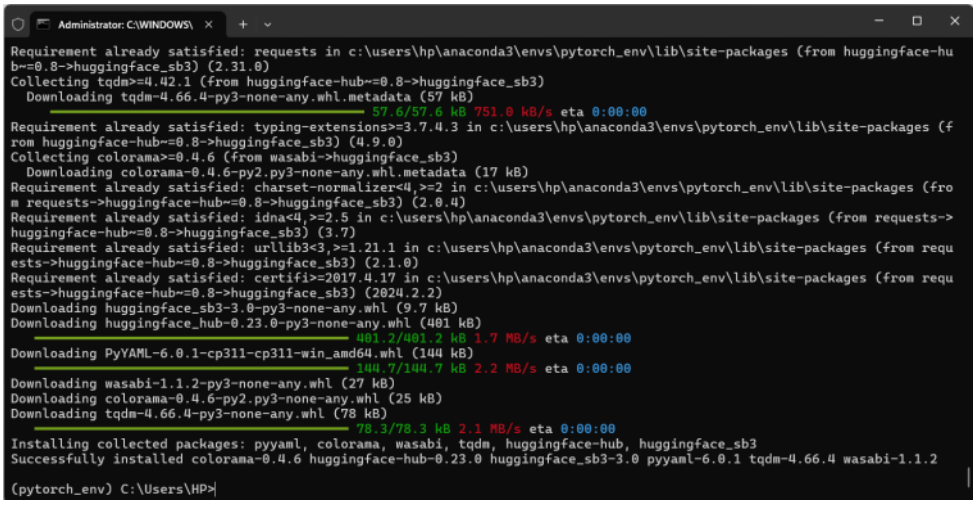
ii. Install visual studio build tools





e. Install Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations with pytorch. huggingface_sb3: Additional code for Stable-baselines3 to load and upload models from the Hugging Face Hub

pip install stable-baselines3==2.0.0a5 swig gymnasium[box2d]huggingface_sb3

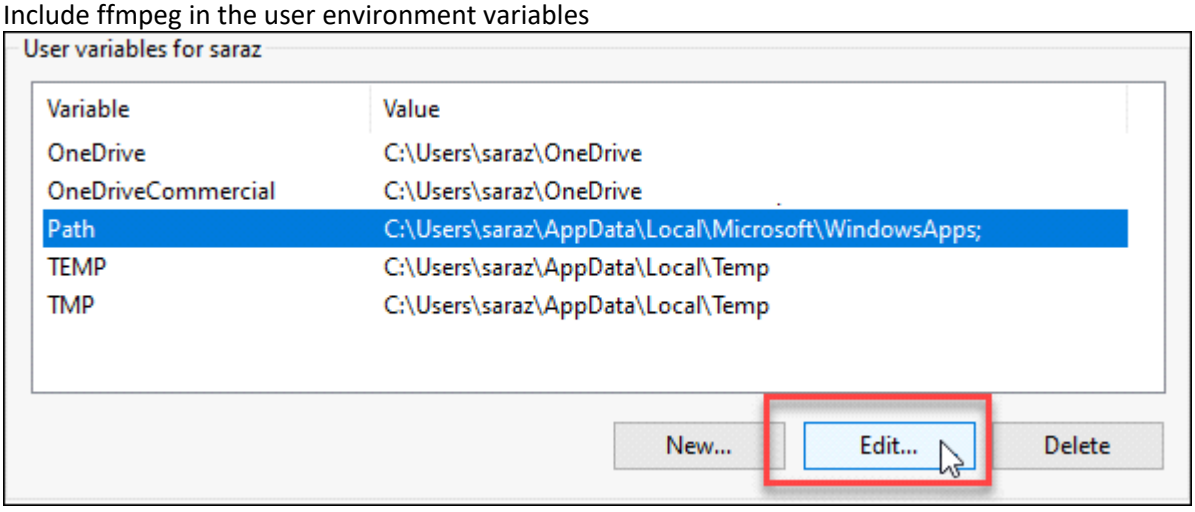


f. Install Virtual-Screen and all prerequisites

```
sudo apt-get update
apt install python-opengl
```

```
apt install ffmpeg
apt install xvfb
pip3 install pyvirtualdisplay
```

g. Include ffmpeg in the user environment variables



2. Import Gymnasium huggingface_sb3 and stable_baseline

```
import gymnasium

from huggingface_sb3 import load_from_hub, package_to_hub
from huggingface_hub import (
    notebook_login,
)  # To log to our Hugging Face account to be able to upload models to the Hub.

from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor
```

3. Understanding Gymnasium and how it works
   a. The library containing our environment is called Gymnasium. You'll use Gymnasium a lot in Deep Reinforcement Learning.
   b. Gymnasium is the new version of Gym library maintained by the Farama Foundation.
   c. The Gymnasium library provides two things:
      i. An interface that allows you to create RL environments.
      ii. A collection of environments (gym-control, atari, box2D…).
   d. Reminder to how Reinforcement Learning works by example

At each step:

- Our Agent receives a **state (S0)** from the **Environment** — we receive the first

frame of our game (Environment).
- Based on that **state (S0),** the Agent takes an **action (A0)** — our Agent will move to the right.
- The environment transitions to a **new state (S1)** — new frame.
- The environment gives some **reward (R1)** to the Agent — we're not dead *(Positive Reward +1)*.

e. How Gymnasium giving us an environment for Reinforcement Learning:
   i. Get an action using our model (in our example we take a random action)
   ii. Using env.step(action), we perform this action in the environment and get
      - observation: The new state (st+1)
      - reward: The reward we get after executing the action
      - terminated: Indicates if the episode terminated (agent reach the terminal state)
      - truncated: Introduced with this new version, it indicates a timelimit or if an agent go out of bounds of the environment for instance.
      - info: A dictionary that provides additional information (depends on the environment).
   iii. We reset the environment to its initial state with observation = env.reset()

4. Create Environment
   a. Create environment with Gymnasium

```
# We create our environment with gym.make("<name_of_the_environment>")
env = gym.make("LunarLander-v2")
env.reset()
print("_____OBSERVATION SPACE_____ \n")
print("Observation Space Shape", env.observation_space.shape)
print("Sample observation", env.observation_space.sample())  # Get a random
observation
```

We see with Observation Space Shape (8,) that the observation is a vector of size 8, where each value contains different information about the lander:
- Horizontal pad coordinate (x)
- Vertical pad coordinate (y)
- Horizontal speed (x)
- Vertical speed (y)
- Angle
- Angular speed
- If the left leg contact point has touched the land (boolean)
- If the right leg contact point has touched the land (boolean)

   b. Create the action space

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

The action space (the set of possible actions the agent can take) is discrete with 4 actions available :
- Action 0: Do nothing,
- Action 1: Fire left orientation engine,
- Action 2: Fire the main engine,
- Action 3: Fire right orientation engine.

   c. State and Reward Function
   Reward function (the function that will give a reward at each timestep) :
   After every step a reward is granted. The total reward of an episode is the **sum of the rewards for all the steps within that episode**.
   For each step, the reward:

- Is increased/decreased the **closer**/further the lander is to the landing pad.
- Is increased/decreased the **slower**/faster the lander is moving.
- Is decreased the more the lander is **tilted** (angle not horizontal).
- Is increased by 10 points for each leg that is in contact with the ground.
- Is decreased by 0.03 points each frame a side engine is firing.
- Is decreased by 0.3 points each frame the main engine is firing.

The episode receive an **additional reward of -100 or +100 points for crashing or landing safely respectively.**

An episode is **considered a solution if it scores at least 200 points.**

d. Vectorized Environment

We create a vectorized environment (a method for stacking multiple independent environments into a single environment) of 16 environments, this way, we'll have more diverse experiences during the training.

```
# Create the environment
env = make_vec_env("LunarLander-v2", n_envs=16)
```

5. Create the model
   a. We have studied our environment and we understood the problem: **being able to land the Lunar Lander to the Landing Pad correctly by controlling left, right and main orientation engine**. Now let's build the algorithm we're going to use to solve this Problem .
   b. To do so, we're going to use our first Deep RL library, Stable Base Line 3
   c. SB3 is a set of **reliable implementations of reinforcement learning algorithms in PyTorch**.
   d. To solve this problem, we're going to use SB3 PPO. PPO (aka Proximal Policy Optimization) is one of the SOTA (state of the art) Deep Reinforcement Learning algorithms that you'll study during this course. PPO consist of:
      - *Value-based reinforcement learning method*: learning an action-value function that will tell us the **most valuable action to take given a state and action**.
      - *Policy-based reinforcement learning method*: learning a policy that will **give us a probability distribution over actions**.
   e. Stable-Baseline3 is easy to setup:
      i. You **create your environment** (in our case it was done above)
      ii. You define the **model you want to use and instantiate this model** model = PPO("MlpPolicy")
      iii. You **train the agent** with model.learn and define the number of training timesteps

```
# Create environment
env = gym.make('LunarLander-v2')

# Instantiate the agent
model = PPO('MlpPolicy', env, verbose=1)
# Train the agent
model.learn(total_timesteps=int(2e5))

# TODO: Define a PPO MlpPolicy architecture
# We use MultiLayerPerceptron (MLPPolicy) because the input is a vector,
# if we had frames as input we would use CnnPolicy
model = PPO(
   policy="MlpPolicy",
   env=env,
   n_steps=1024,
   batch_size=64,
   n_epochs=4,
   gamma=0.999,
```

```
    gae_lambda=0.98,
    ent_coef=0.01,
    verbose=1,
)
```

6. Train the PPO agent
   Let's train our agent for 1,000,000 timesteps, don't forget to use GPU on It will take approximately ~20min, but you can use fewer timesteps if you just want to try it out.

```
# Train it for 1,000,000 timesteps
model.learn(total_timesteps=1000000)
# Save the model
model_name = "ppo-LunarLander-v2"
model.save(model_name)
```

7. Evaluate the agent
   a. Remember to wrap the environment in a monitor.
   b. Now that our Lunar Lander agent is trained , we need to **checkits performance**.
   c. Stable-Baselines3 provides a method to do that: evaluate_policy.
   d. In the next step, we'll see **how to automatically evaluate and share your agent to compete in a leaderboard, but for now let's do it ourselves**

```
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10,
deterministic=True)
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

8. Full code alternative 1

```
# -*- coding: utf-8 -*-
"""
Created on Sat May  4 17:13:20 2024

@author: HP
"""
#Import Gymnasium, Stable_baseline, and Huggingface_sb3

import gymnasium as gym

from huggingface_sb3 import load_from_hub, package_to_hub
from huggingface_hub import (
    notebook_login,
)  # To log to our Hugging Face account to be able to upload models to the Hub.

from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor


# We create our environment with gym.make("LunarLander-v2")
env = gym.make("LunarLander-v2")
# Create the environment
env = make_vec_env("LunarLander-v2", n_envs=16)

env.reset()
print("_____OBSERVATION SPACE_____ \n")
```

```
print("Observation Space Shape", env.observation_space.shape)
print("Sample observation", env.observation_space.sample())  # Get a random observation

# Instantiate the agent
model = PPO('MlpPolicy', env, verbose=1)
# Train the agent
model.learn(total_timesteps=int(2e5))

# Define a PPO MlpPolicy architecture
# We use MultiLayerPerceptron (MLPPolicy) because the input is a vector,
model = PPO(
    policy="MlpPolicy",
    env=env,
    n_steps=1024,
    batch_size=64,
    n_epochs=4,
    gamma=0.999,
    gae_lambda=0.98,
    ent_coef=0.01,
    verbose=1,
)

# Train it for 1,000,000 timesteps
model.learn(total_timesteps=1000000)
# Save the model
model_name = "ppo-LunarLander-v2"
model.save(model_name)

eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10,
deterministic=True)
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

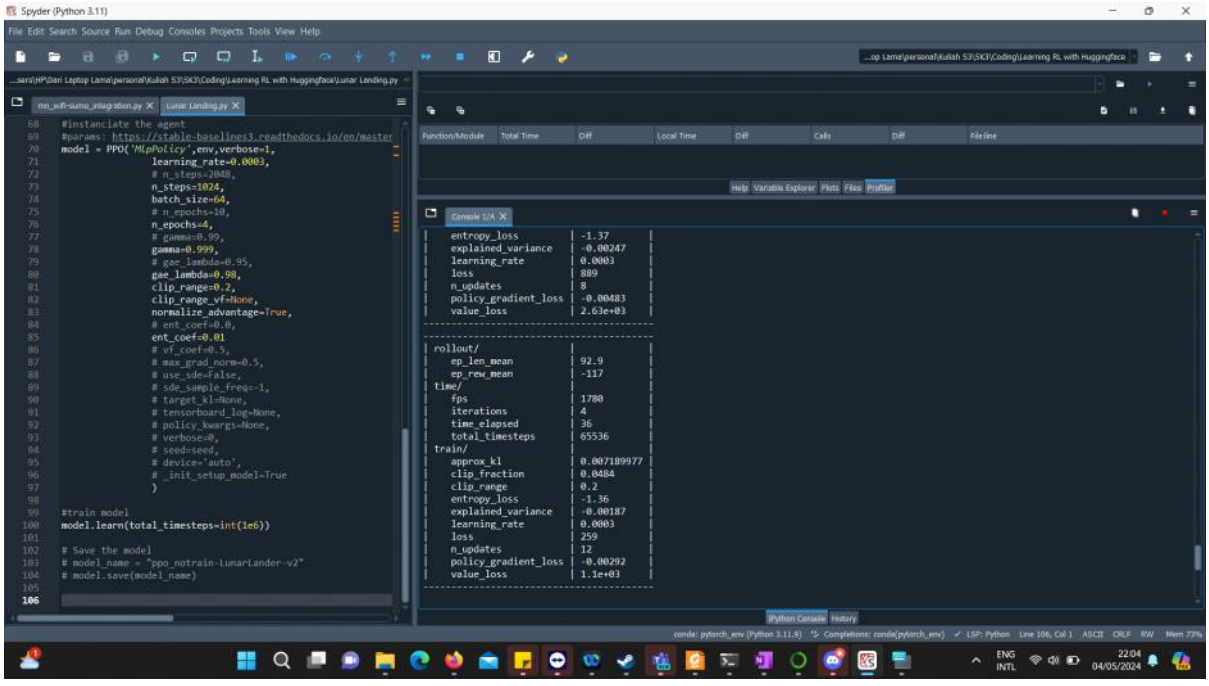
9. Fullcode Alternative 2

```
#imports
import gym
from stable_baselines3 import PPO
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.vec_env import DummyVecEnv
from huggingface_sb3 import load_from_hub, package_to_hub, push_to_hub
from huggingface_hub import (
    notebook_login,
)

# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900));
virtual_display.start();

#create seed
seed=11

%%time
```

```
#@title create and train the model
#create enviroment
env = gym.make('LunarLander-v2')

#reset enviroment ot initial state
env.reset()

#create vectorized enviroment
env = make_vec_env("LunarLander-v2", n_envs=16)

#instanciate the agent
#params: https://stable-
baselines3.readthedocs.io/en/master/modules/ppo.html#parameters
model = PPO('MlpPolicy',env,verbose=1,
        learning_rate=0.0003,
        # n_steps=2048,
        n_steps=1024,
        batch_size=64,
        # n_epochs=10,
        n_epochs=4,
        # gamma=0.99,
        gamma=0.999,
        # gae_lambda=0.95,
        gae_lambda=0.98,
        clip_range=0.2,
        clip_range_vf=None,
        normalize_advantage=True,
        # ent_coef=0.0,
        ent_coef=0.01
        # vf_coef=0.5,
        # max_grad_norm=0.5,
        # use_sde=False,
        # sde_sample_freq=-1,
        # target_kl=None,
        # tensorboard_log=None,
        # policy_kwargs=None,
        # verbose=0,
        # seed=seed,
        # device='auto',
        # _init_setup_model=True
        )

#train model
model.learn(total_timesteps=int(1e6))

# Save the model
# model_name = "ppo_notrain-LunarLander-v2"
# model.save(model_name)
```

10. Alternative 3 using GPU to running

```
# imports
import gym
from stable_baselines3 import PPO
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.env_util import make_vec_env
```

```
from stable_baselines3.common.vec_env import VecNormalize

# Create seed
seed = 11

# Title create and train the model
# Create environment
env = gym.make('LunarLander-v2')

# Reset environment to initial state
env.reset()

# Create vectorized environment
env = make_vec_env("LunarLander-v2", n_envs=16)

# Wrap environment to normalize observations and rewards
env = VecNormalize(env, norm_obs=True, norm_reward=True, clip_obs=10.)

# Instantiate the agent with CUDA support
model = PPO('MlpPolicy', env, verbose=1, device='cuda')

# Train model
model.learn(total_timesteps=int(1e6))

# Save the model
# model_name = "ppo_notrain-LunarLander-v2"
# model.save(model_name)
```

11. Run the program